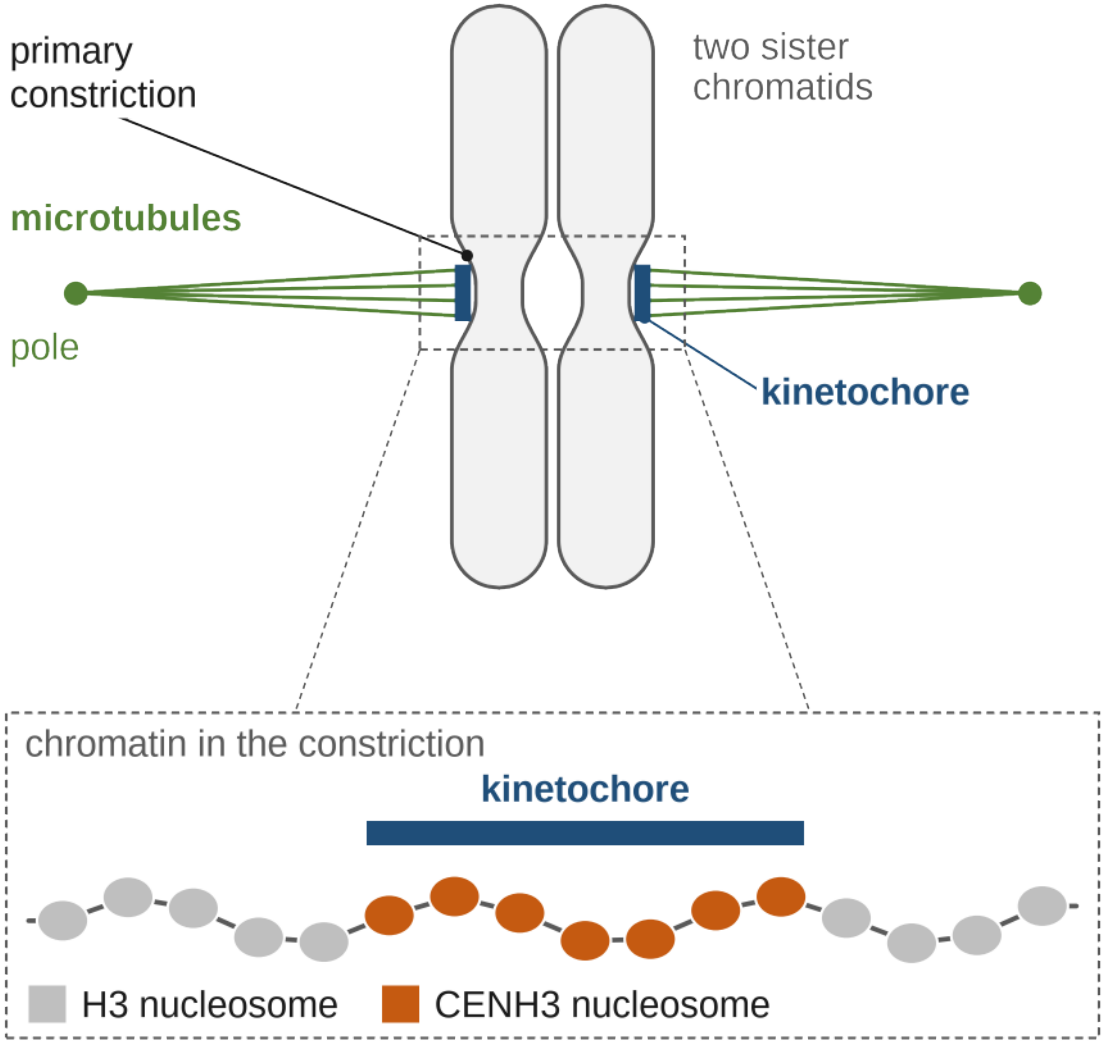
The biology and the experiment

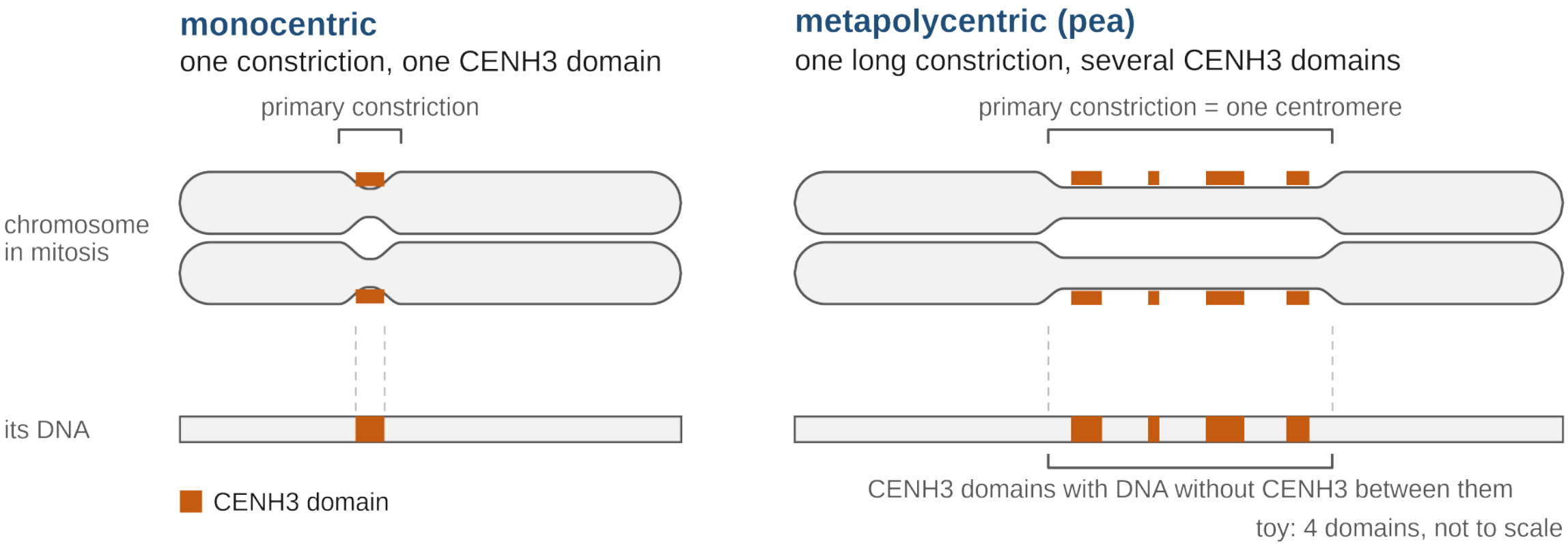
The centromere: where the spindle pulls

- Centromere = where the **kinetochore** forms and **microtubules** attach in mitosis
- Visible as the **primary constriction** of a metaphase chromosome
- Not defined by a DNA sequence, but by a histone
- Centromeric nucleosomes carry **CENH3** instead of histone H3
- Find CENH3 → find the centromere (ChIP-seq)



Allshire & Karpen 2008, Nat Rev Genet 9:923; Neumann et al. 2012, PLoS Genet 8:e1002777

One centromere, many CENH3 domains



pea chromosome 6: how many domains, where, how large? → Part 1

Neumann et al. 2012, PLoS Genet 8:e1002777; Macas et al. 2023, PLoS Genet 19:e1010633

Satellite DNA: one sequence, repeated

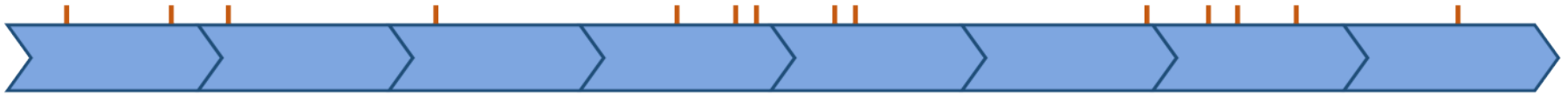
1 one monomer



TGGTGTTAACCTTACTATACTCCCGCTCCGGGGTTTGGCT

toy monomer: 40 bp
real monomers: typically 150–2,000 bp

2 an array: the monomer repeated head-to-tail (orange tick = a base different from the monomer)



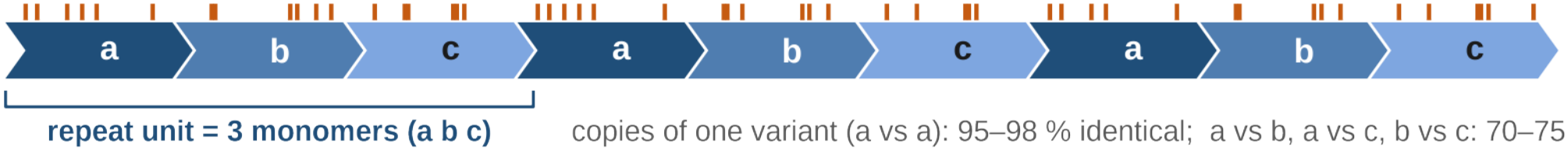
SatA

95.0 % 97.5 % 97.5 % 92.5 % 95.0 % 97.5 % 92.5 % 97.5 %

← identity to the monomer

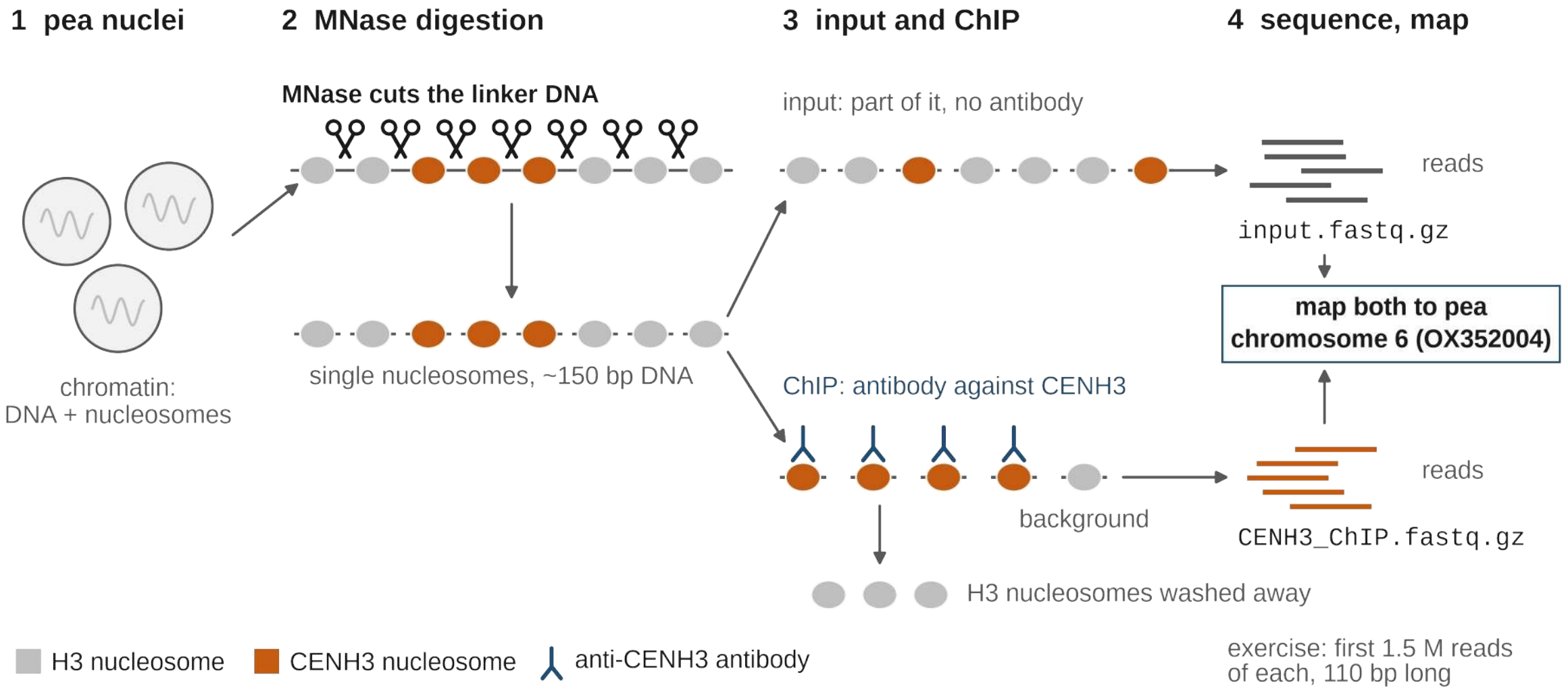
copies similar, not identical; real arrays: thousands of copies, kb to Mb long

3 a higher-order repeat: a block of different monomers repeated as a unit



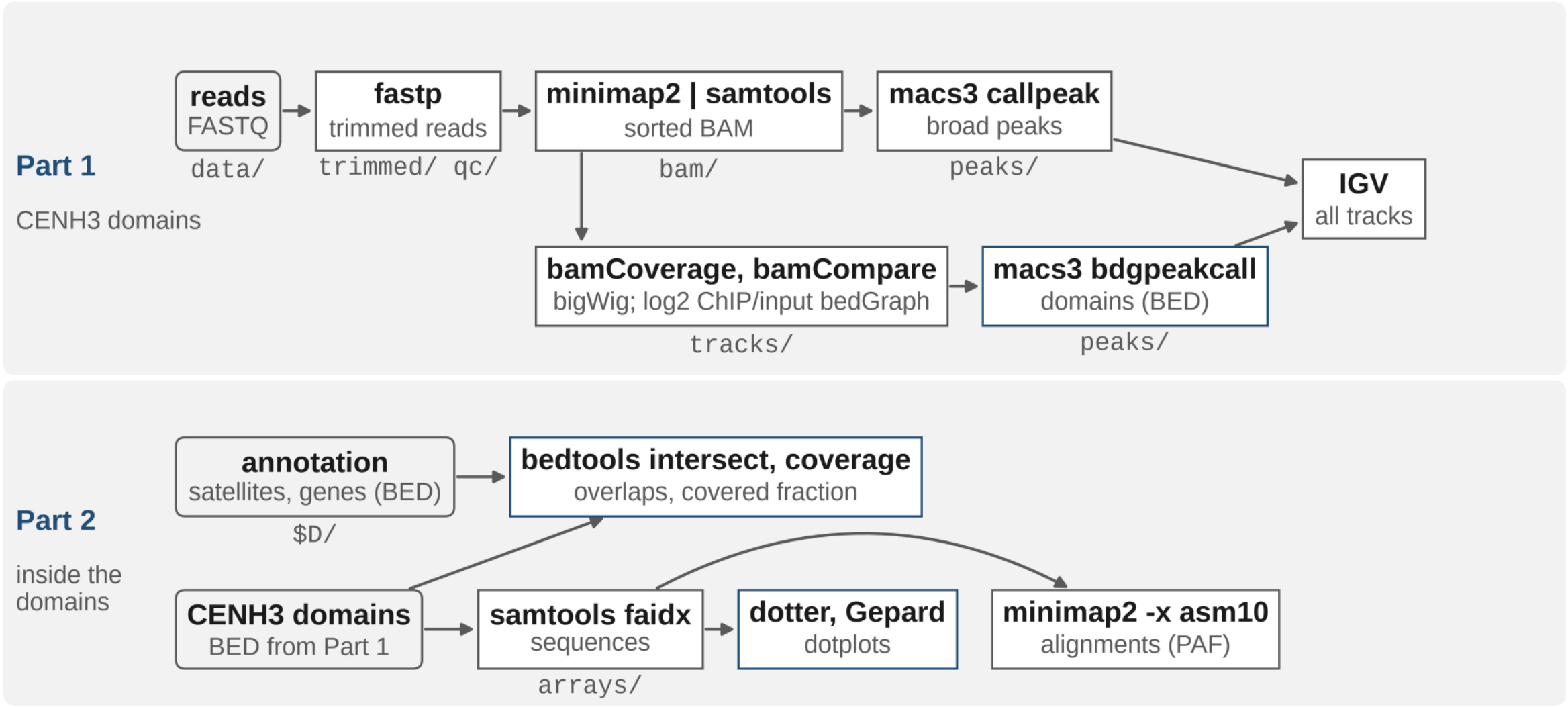
copies of one variant (a vs a): 95–98 % identical; a vs b, a vs c, b vs c: 70–75 %

The experiment: native ChIP with MNase



Macas et al. 2023, PLoS Genet 19:e1010633 (ENA PRJEB54858); Neumann et al. 2012, PLoS Genet 8:e1002777 (ChIP protocol)

Two parts, one chromosome



The command line you will use

Anatomy of a command

program = what runs

option name: starts with -

option value: follows its name

```
fastp -i data/CENH3_ChIP.fastq.gz \
      -o trimmed/CENH3_ChIP.fastq.gz \
      -f 9 \
      -l 80 \
      -w 4 \
      -h qc/CENH3_ChIP.html \
      -j qc/CENH3_ChIP.json
```

- input: the raw reads
- output: the trimmed reads
- **remove the first 9 bases of each read**
- **drop reads shorter than 80 bases**
- 4 threads (CPU cores) at once
- output: report for you (web page)
- output: report for programs

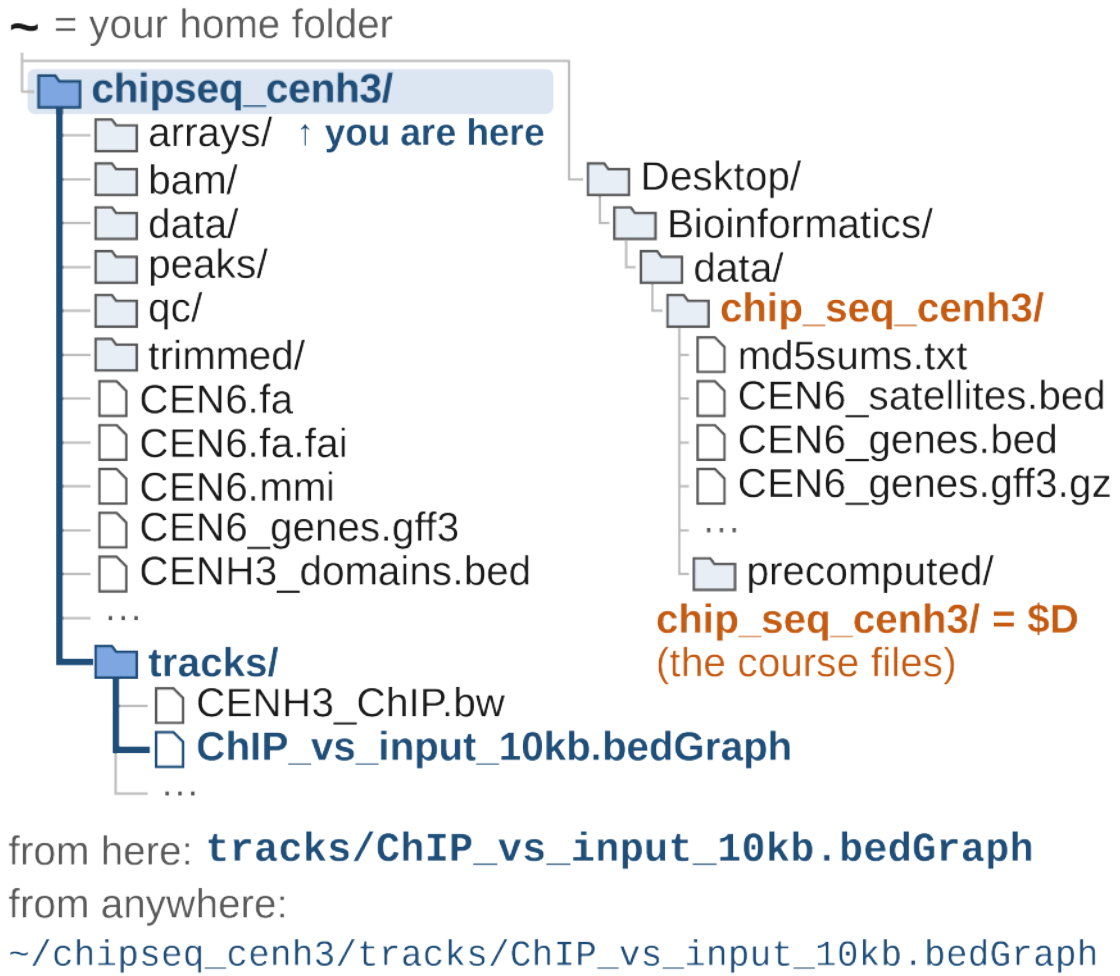
```
program  -option value  -option value  ...
```

the pattern of every command

- spaces separate the parts
- `\` at the end of a line: the command goes on (in the exercise it is one line)
- **`fastp --help`** lists all options of fastp

Where am I? Paths and folders

- The terminal always works **in one folder** (`pwd`)
- `cd ~/chipseq_cenh3` moves there; `~` = your home
- `bam/input.bam` = file in a sub-folder
- `mkdir -p bam` makes the folder
- `ls bam` lists it



from here: **`tracks/ChIP_vs_input_10kb.bedGraph`**

from anywhere:
`~/chipseq_cenh3/tracks/ChIP_vs_input_10kb.bedGraph`

Variables: a name for a long path

1 store

```
D=~/Desktop/Bioinformatics/data/chip_seq_cenh3
```

name = with no spaces around it — **the text to store**

memory of the terminal

D	/home/student/Desktop/Bioinformatics/data/chip_seq_cenh3

~ is already replaced by your home folder

2 use

```
wc -l $D/CEN6_genes.bed
```

← you type

the shell replaces $D by the stored text, then runs:

```
wc -l /home/student/Desktop/Bioinformatics/data/chip_seq_cenh3/CEN6_genes.bed
```

(student = your user name)

echo $D shows the stored text

closing the terminal forgets D → type the D= line again

the same for `Z=https://zenodo.org/records/23988194/files` used as `wget -c $Z/$f`

for loops: one command, many samples

```
for s in CENH3_ChIP input; do
    minimap2 -ax sr -t 4 CEN6.mmi trimmed/$s.fastq.gz \
      | samtools sort -@ 2 -o bam/$s.bam
    samtools index bam/$s.bam
done
```

s = each value of the list, one after the other

2 commands repeated; $s = the current value

done = end of the loop

what the shell runs, round by round:

s = CENH3_ChIP

```
minimap2 -ax sr -t 4 CEN6.mmi trimmed/CENH3_ChIP.fastq.gz \
  | samtools sort -@ 2 -o bam/CENH3_ChIP.bam
samtools index bam/CENH3_ChIP.bam
```

s = input

```
minimap2 -ax sr -t 4 CEN6.mmi trimmed/input.fastq.gz \
  | samtools sort -@ 2 -o bam/input.bam
samtools index bam/input.bam
```

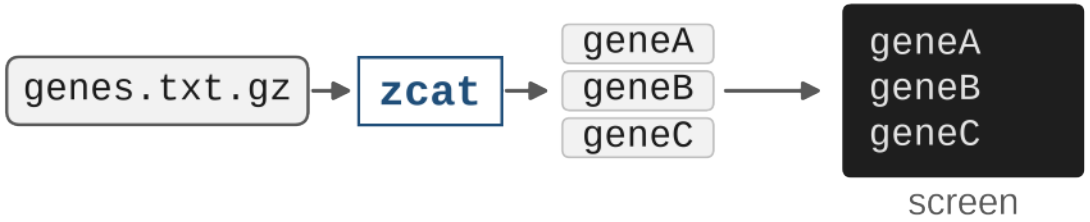
```
$ for s in CENH3_ChIP input; do
> 
```

> = the loop is not finished:
the shell waits for done

Where does the output go?

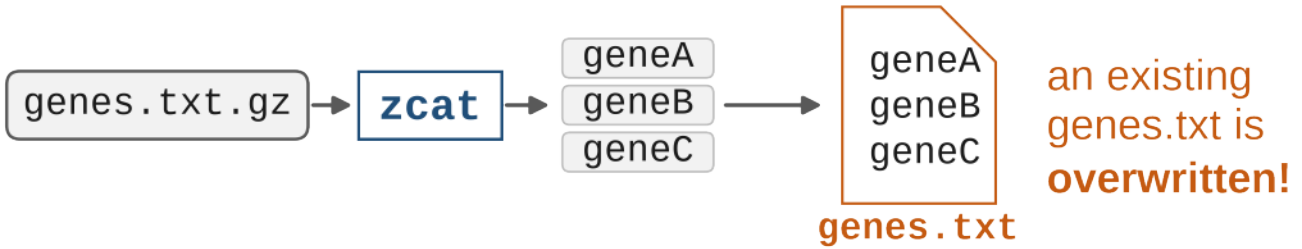
```
$ zcat genes.txt.gz
geneA
geneB
geneC
$
```

no redirection: the text goes to the screen



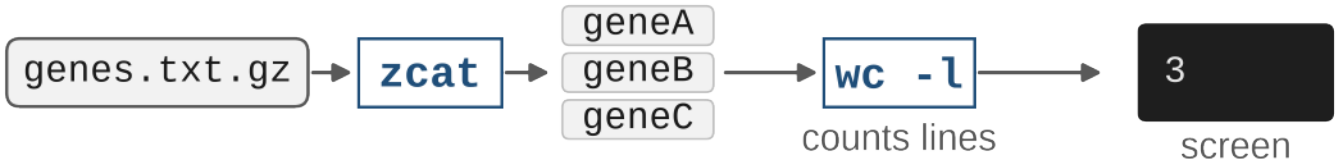

```
$ zcat genes.txt.gz > genes.txt
$
```

> file: the text goes into a file, the screen stays empty

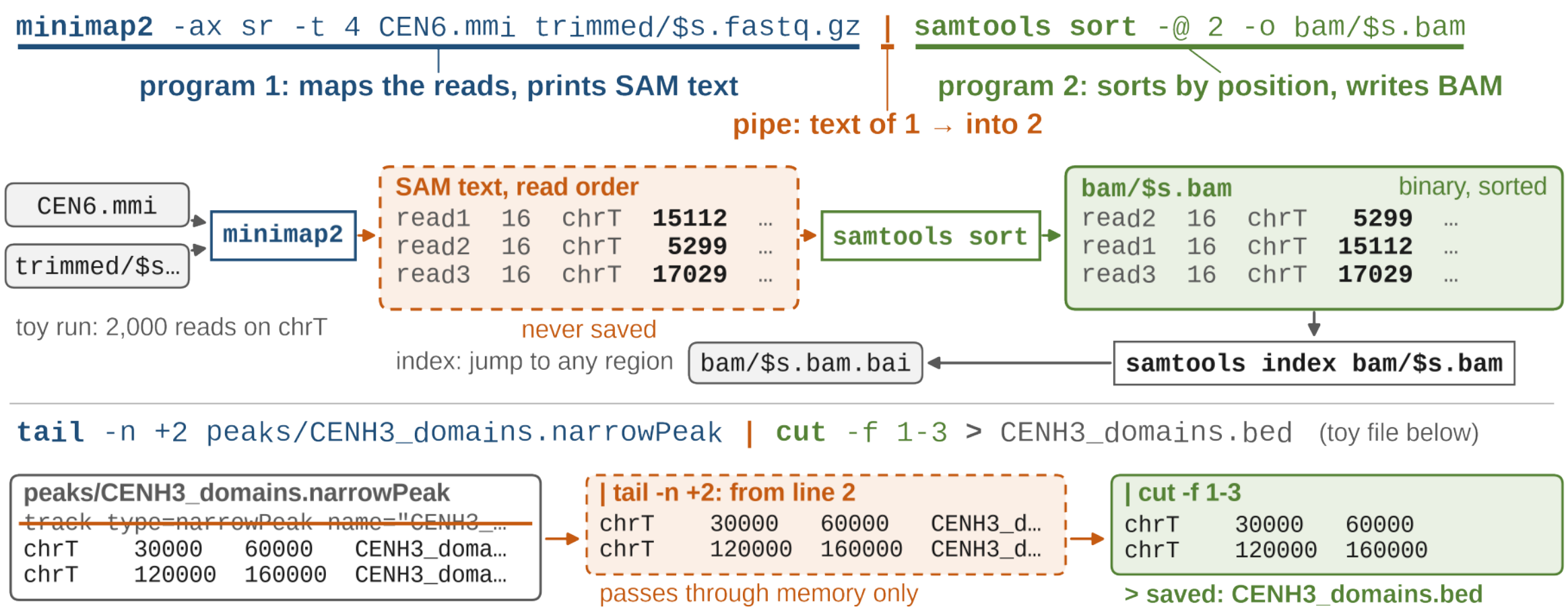



```
$ zcat genes.txt.gz | wc -l
3
$
```

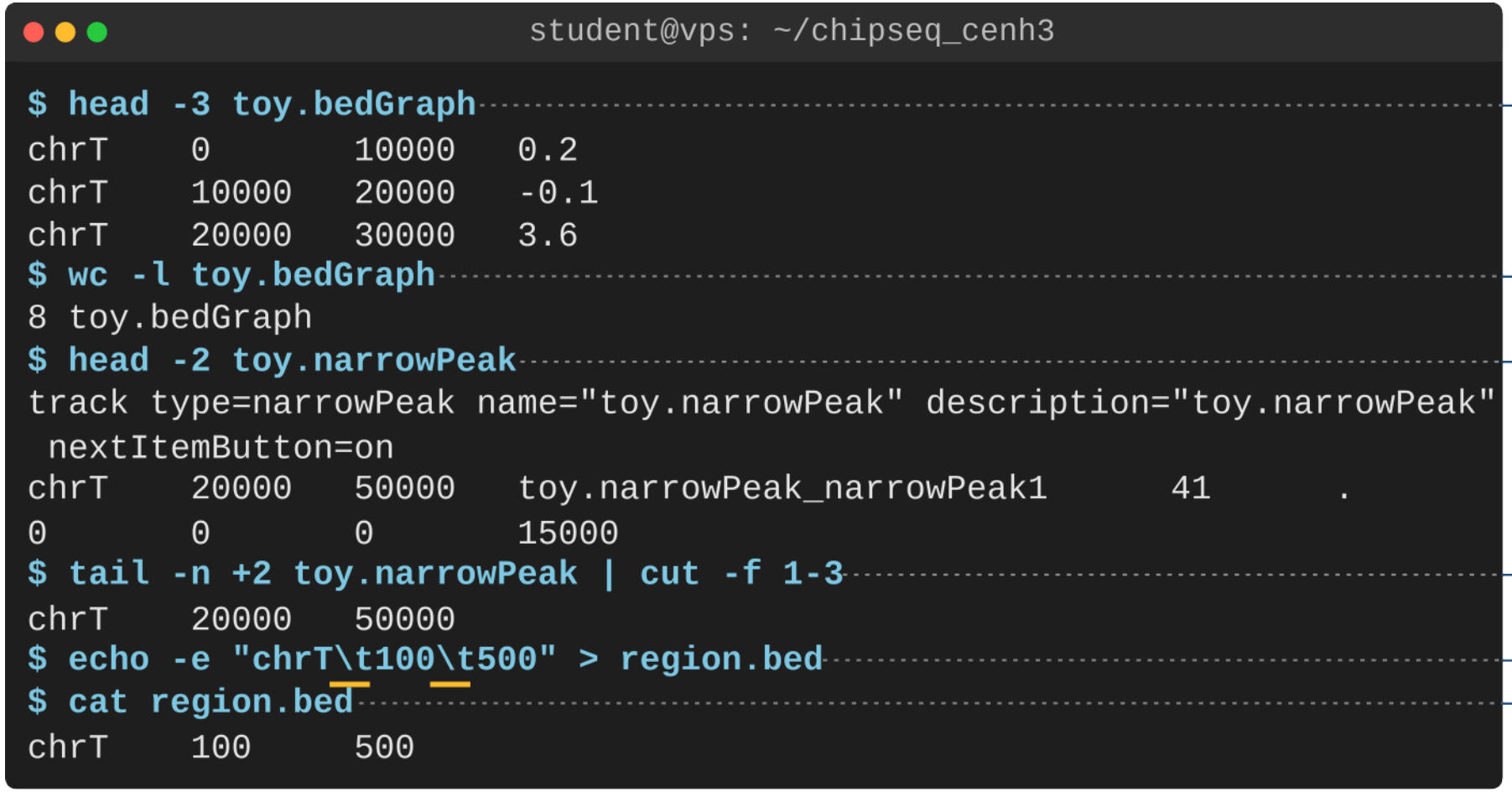
| program: the text flows into the next program



Pipes: no file in between

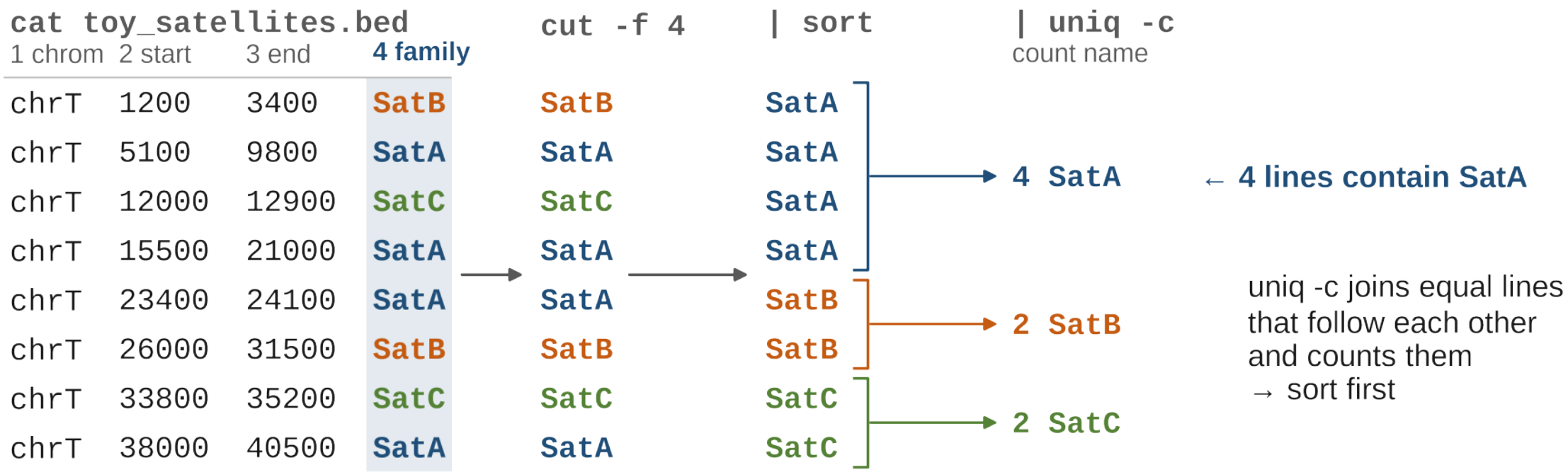


Looking into files



Columns: cut, sort, uniq -c

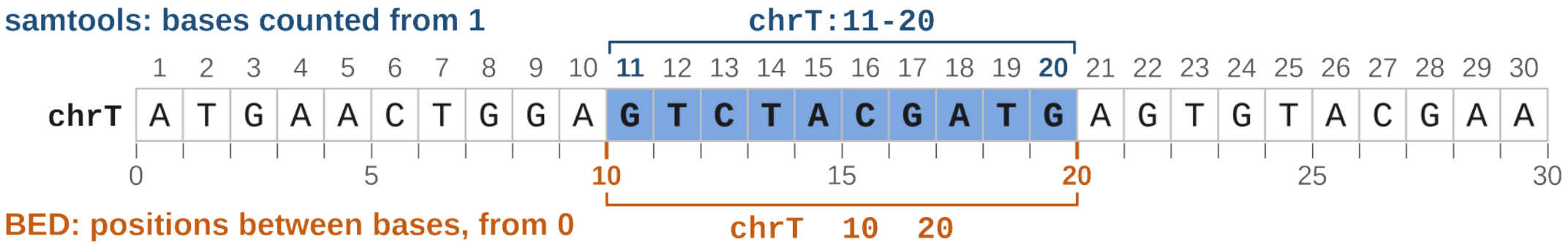
```
cut -f 4 toy_satellites.bed | sort | uniq -c
```



in the exercise (2.1), the arrays that overlap a domain:

```
bedtools intersect -a $D/CEN6_satellites.bed -b CENH3_domains.bed -u | cut -f 4 | sort | uniq -c
```

Two ways to write a position



student@vps: ~/chipseq_cenh3

```
$ samtools faidx toy.fa chrT:11-20
>chrT:11-20
GTCTACGATG
```

1-based, both ends included

(as in the exercise: OX352004:84800001-84820000)

student@vps: ~/chipseq_cenh3

```
$ cat region.bed
chrT    10      20
$ bedtools getfasta -fi toy.fa -bed region.bed
>chrT:10-20
GTCTACGATG
```

start = bases before the region; end included

the same 10 bases (in yellow)

Was the download damaged?

- `md5sum file` computes a 32-character **checksum**
- One changed byte → a completely different checksum
- `md5sum -c md5sums.txt` checks all files: `OK` or `FAILED`
- `FAILED` → delete the file, download it again

sampleB.fastq (published)

```
@read_B1
AAAACCATGTCCGTAA
+
FFFIIIIIIFFIIIFF
```

sampleB.fastq (downloaded)

```
@read_B1
AAAACCATGTCCGTGA
+
FFFIIIIIIFFIIIFF
```

one base changed

published **64b23c2073957478d9e2de82b2bb169f**

downloaded **dbb3ed034eafdc56169bddd9b7a25f0e**

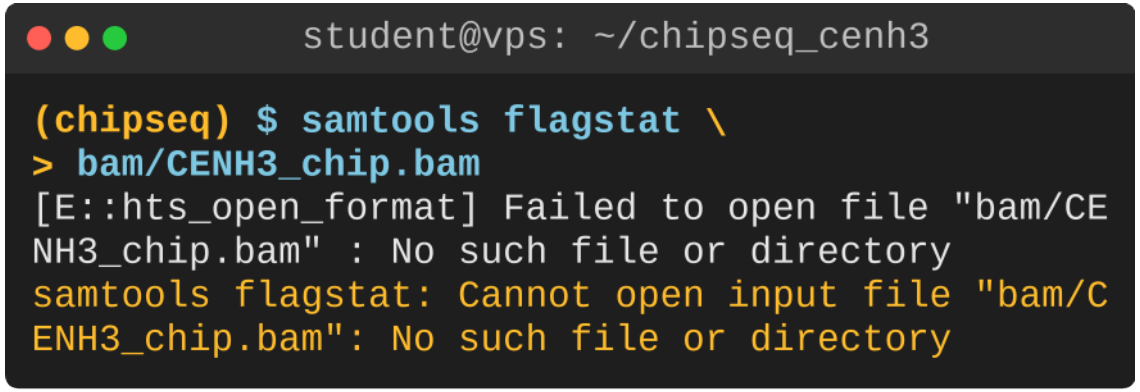
29 of the 32 characters differ

```
student@vps: ~/chipseq_abf3

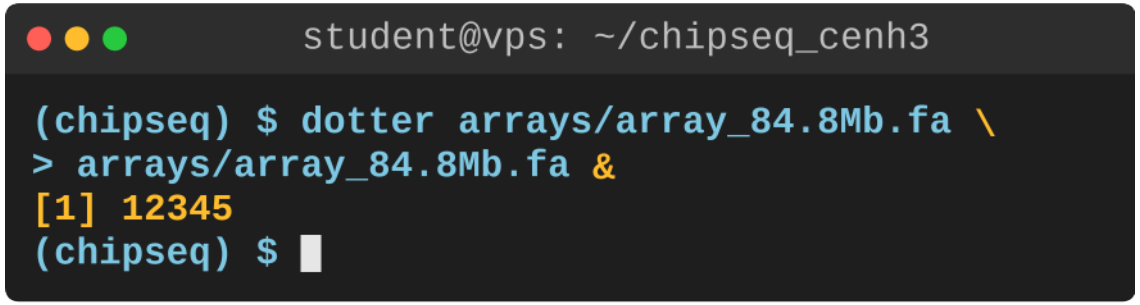
$ cat md5sums.txt
d083a4bf4226a8f2bde0e496f9e13248  sampleA.fastq
64b23c2073957478d9e2de82b2bb169f  sampleB.fastq
58f1afdd0ccdf1706732b1a1f73189f8  sampleC.fastq
$ md5sum -c md5sums.txt
sampleA.fastq: OK
sampleB.fastq: FAILED
sampleC.fastq: OK
md5sum: WARNING: 1 computed checksum did NOT mat
ch
```

Survival tips

- **Tab** completes names, ↑ recalls commands
- **Ctrl+C** stops a running program
- `\` at the line end: the command goes on
- `&` at the end: a window program (dotter) runs, the terminal stays free
- `conda activate chipseq` in each new terminal
- Error? Read the **last line** first



(chipseq) = the conda env is active
\ and > = the command goes on
last line: what went wrong
CENH3_chip ≠ CENH3_ChIP: case matters



& = dotter opens its window,
[1] 12345 = job number, process ID;
the prompt is back at once

Part 1 – From reads to CENH3 domains

fastp: trimming and the base content

`fastp -f 9 -l 8` (toy; exercise: -l 80)

-f 9: first 9 bases cut off

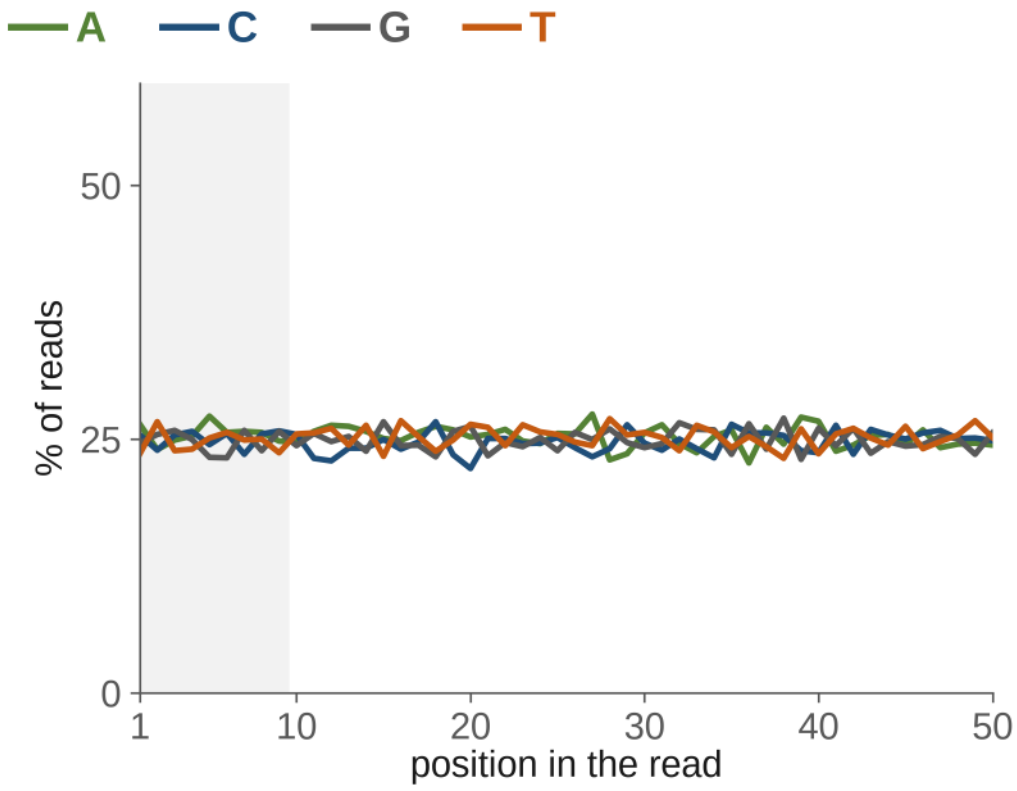
read1	TGGGCGAAC	**TTGGTCACCCC**	11 bases kept
read2	GAAGTATCT	**GATGAGATGAT**	11 bases kept
read3	CACCGAGAG	**CCGGGGCGAGG**	11 bases kept
read4	AAGATGTAC	~~**GGATAC**~~	**6 left < 8: dropped**
read5	TTTCCGCAC	**AGGGACTAGGT**	11 bases kept
read6	TAACCGCGA	**TTTCTTATCCT**	11 bases kept

5 of 6 reads written to the trimmed file

Before filtering: read1: base contents

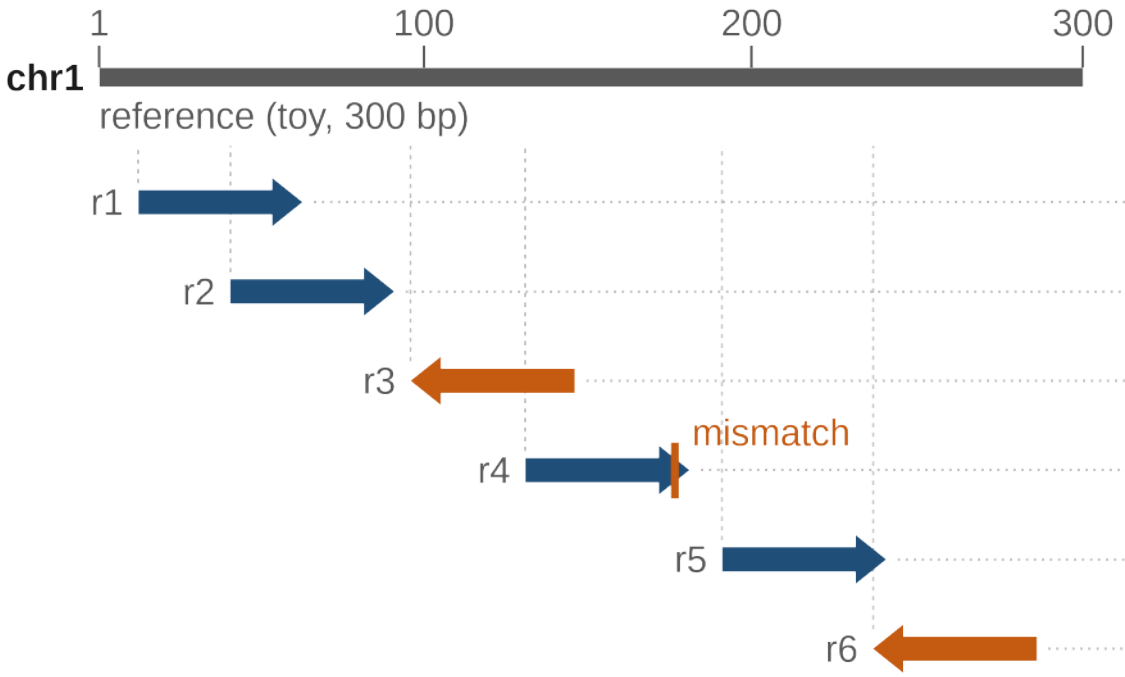
toy: 2,000 random reads of 50 bases

your reads: look at the first positions



random library: four flat lines near 25 %

Mapping: reads to positions



QNAME	FLAG	RNAME	POS	MAPQ	CIGAR
r1	0	chr1	13	34	50M
r2	0	chr1	41	13	50M
r3	16	chr1	96	37	50M
r4	0	chr1	131	22	50M
r5	0	chr1	191	26	50M
r6	16	chr1	237	20	50M

FLAG 0 = read on the + strand (→)

FLAG 16 = read on the − strand (←)

POS = first reference base covered by the read

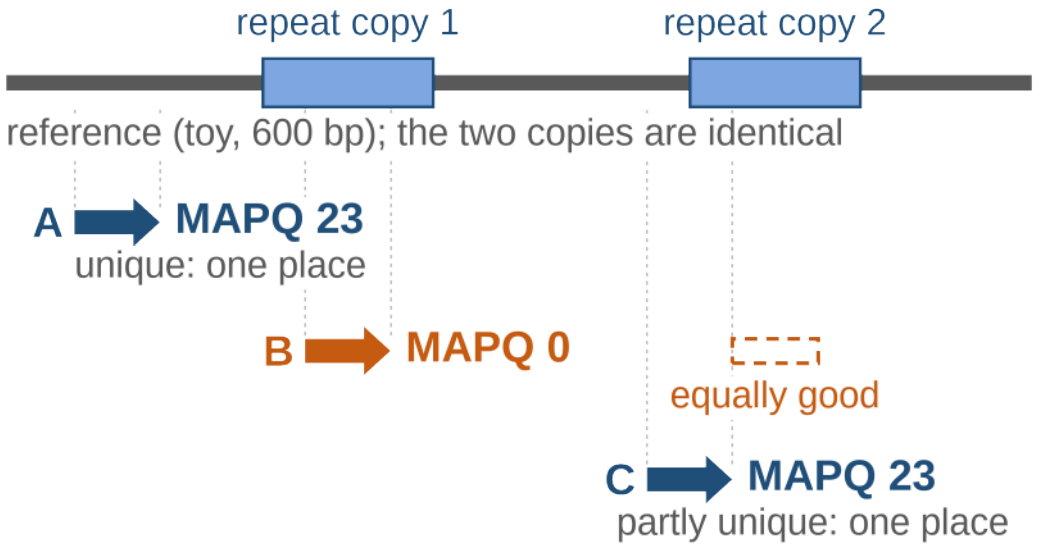
CIGAR 50M = 50 aligned bases

(M = match or mismatch: r4 is 50M too)

MAPQ = how certain the position is

MAPQ: is the position certain?

- A read from a **repeat** fits several places equally
- The mapper picks one at random and reports **MAPQ ≈ 0**
- Unique read → high MAPQ (minimap2 max. 60)
- `-q 10` / `--minMappingQuality 10` keep only reads with MAPQ ≥ 10
- Think before filtering: what is in your regions?

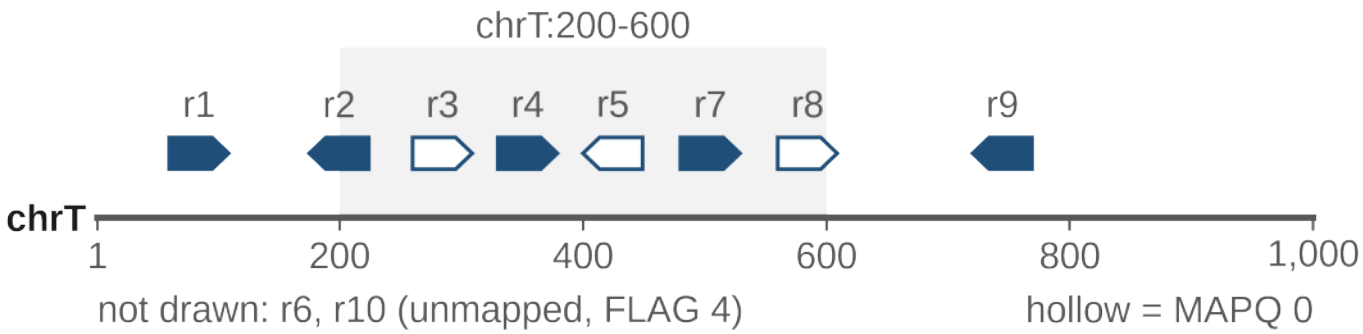


QNAME	FLAG	RNAME	POS	MAPQ	CIGAR
A	0	chr1	41	**23**	50M
B	0	chr1	176	**0**	50M
C	0	chr1	376	**23**	50M

--minMappingQuality 10 keeps A and C, drops B

samtools view -c: counting reads

read	FLAG	POS	MAPQ	1	2	3	4
r1	0	60	60	●	●	●	○
r2	16	175	60	●	●	●	●
r3	0	260	0	●	●	○	●
r4	0	330	60	●	●	●	●
r5	16	400	0	●	●	○	●
r6	4	0	0	●	○	○	○
r7	0	480	60	●	●	●	●
r8	0	560	0	●	●	○	●
r9	16	720	60	●	●	●	○
r10	4	0	0	●	○	○	○
			count	**10**	**8**	**5**	**6**



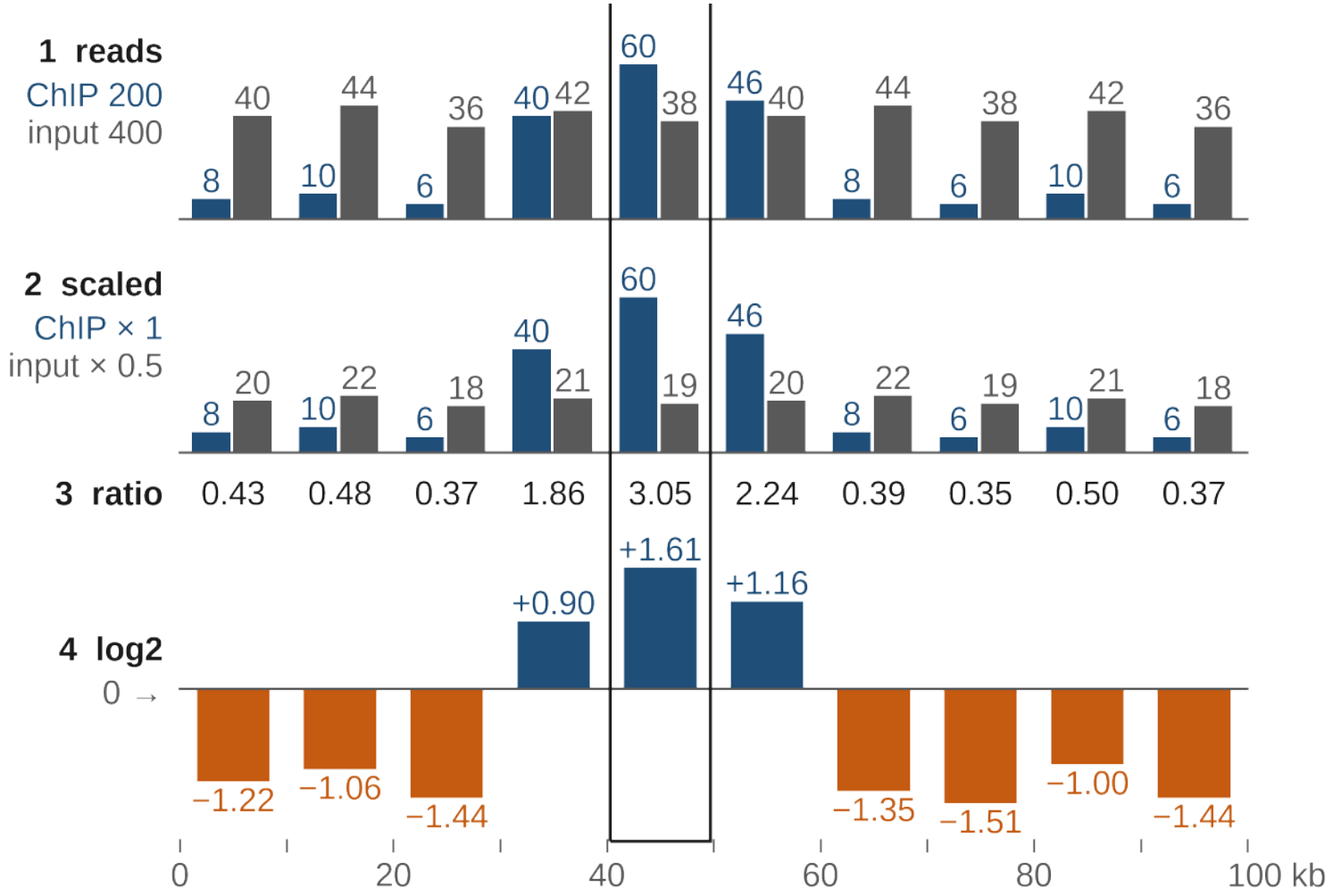
not drawn: r6, r10 (unmapped, FLAG 4)

hollow = MAPQ 0

1. `samtools view -c toy.bam` → **10**
2. `samtools view -c -F 4 toy.bam` → **8**
3. `samtools view -c -F 4 -q 10 toy.bam` → **5**
4. `samtools view -c -F 4 toy.bam chrT:200-600` → **6**

a region counts every read that overlaps it (r2, r8)

ChIP / input in bins: log2 ratio



bin 40–50 kb

reads: ChIP 60, input 38

scaled: input 38 × 0.5 = 19

ratio: (60 + 1) / (19 + 1) = 3.05

log2(3.05) = 1.61

+ 1 = pseudocount: no division by 0

log2(ChIP / input)

- **3** 8 × more than input
- **1** 2 × more
- **0** same as input
- **−1** half of input

toy counts; bamCompare defaults:
`--scaleFactorsMethod readCount`,
`--pseudocount 1`

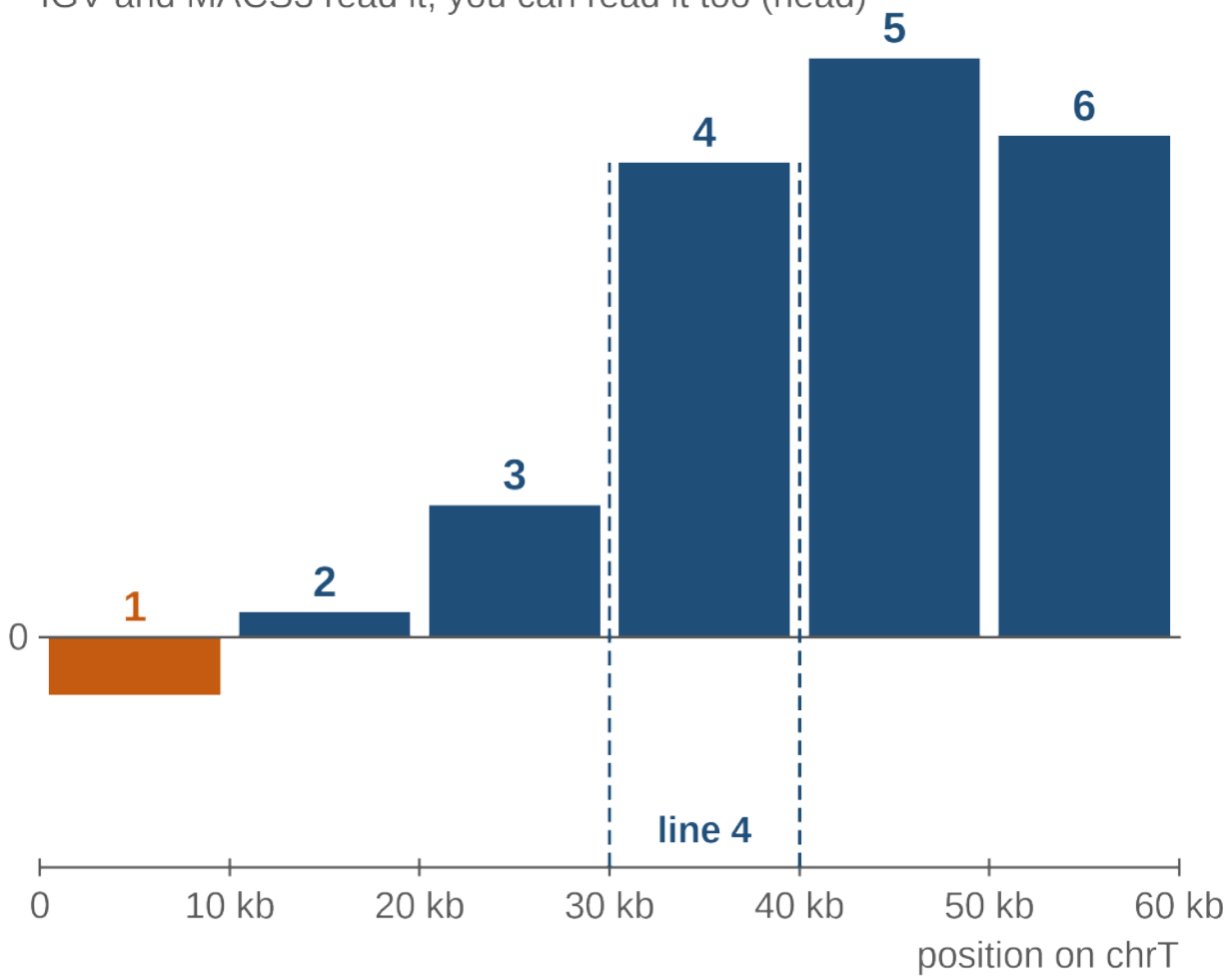
bedGraph: a value for each interval

```
$ head -6 toy.bedGraph
```

	chrom	start	end	value
1	chrT	0	10000	-0.42
2	chrT	10000	20000	0.18
3	chrT	20000	30000	0.95
4	chrT	30000	40000	3.42
5	chrT	40000	50000	4.17
6	chrT	50000	60000	3.61

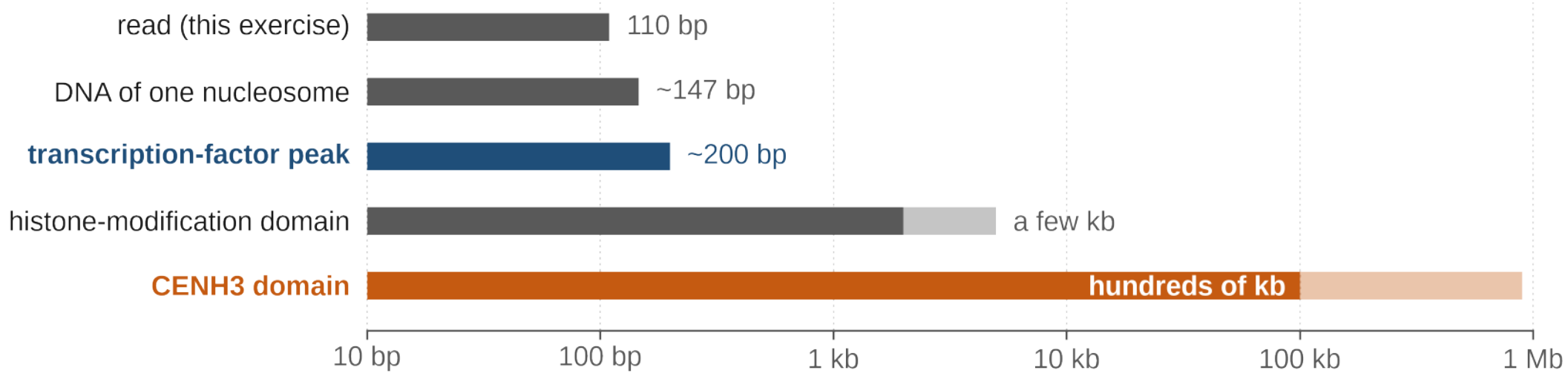
columns separated by TAB
start 0-based, end not included (as BED)

IGV and MACS3 read it; you can read it too (head)

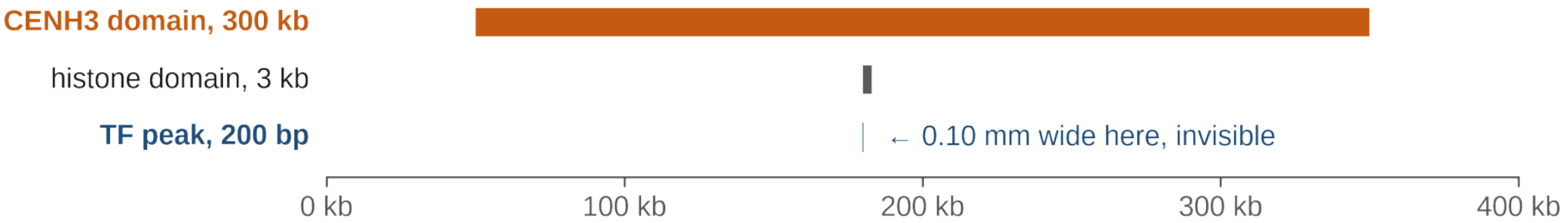


Sizes: a peak is not a domain

log scale: each step = 10×

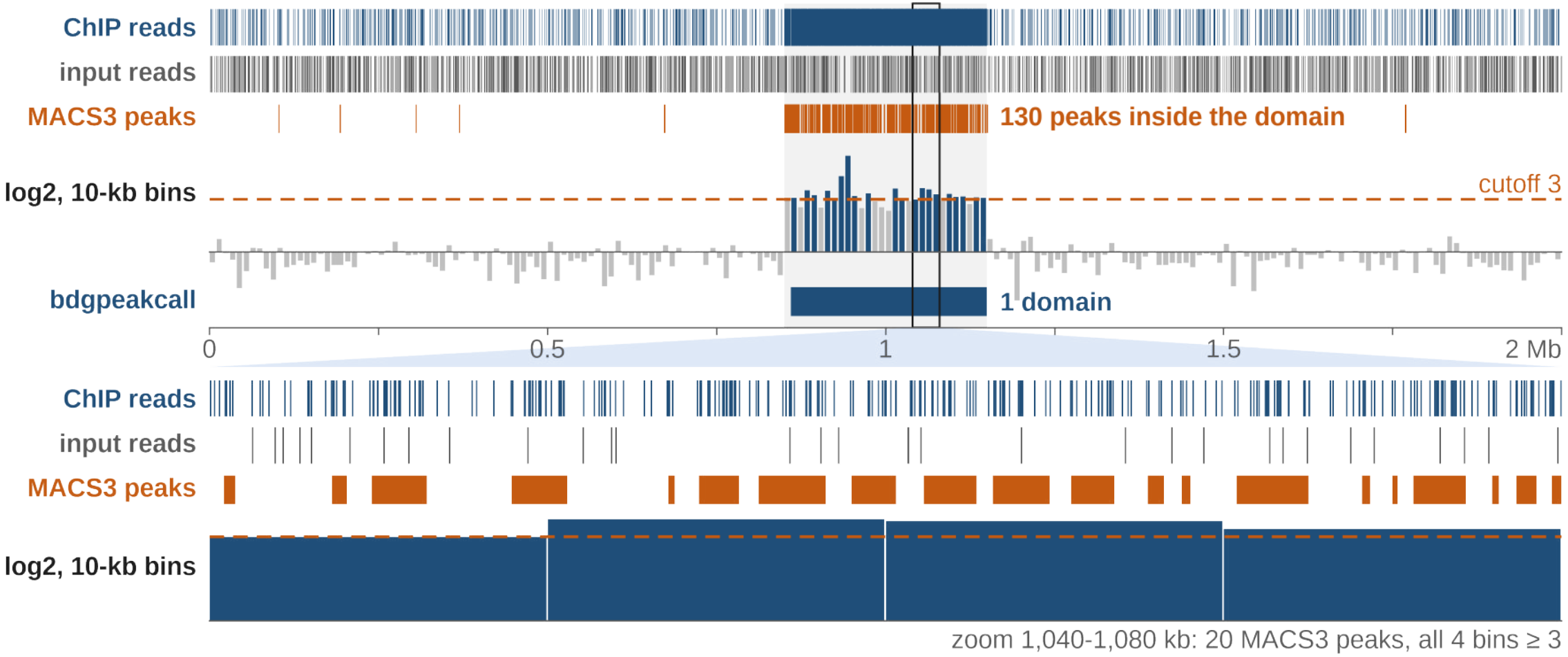


the same on a linear scale (toy sizes)

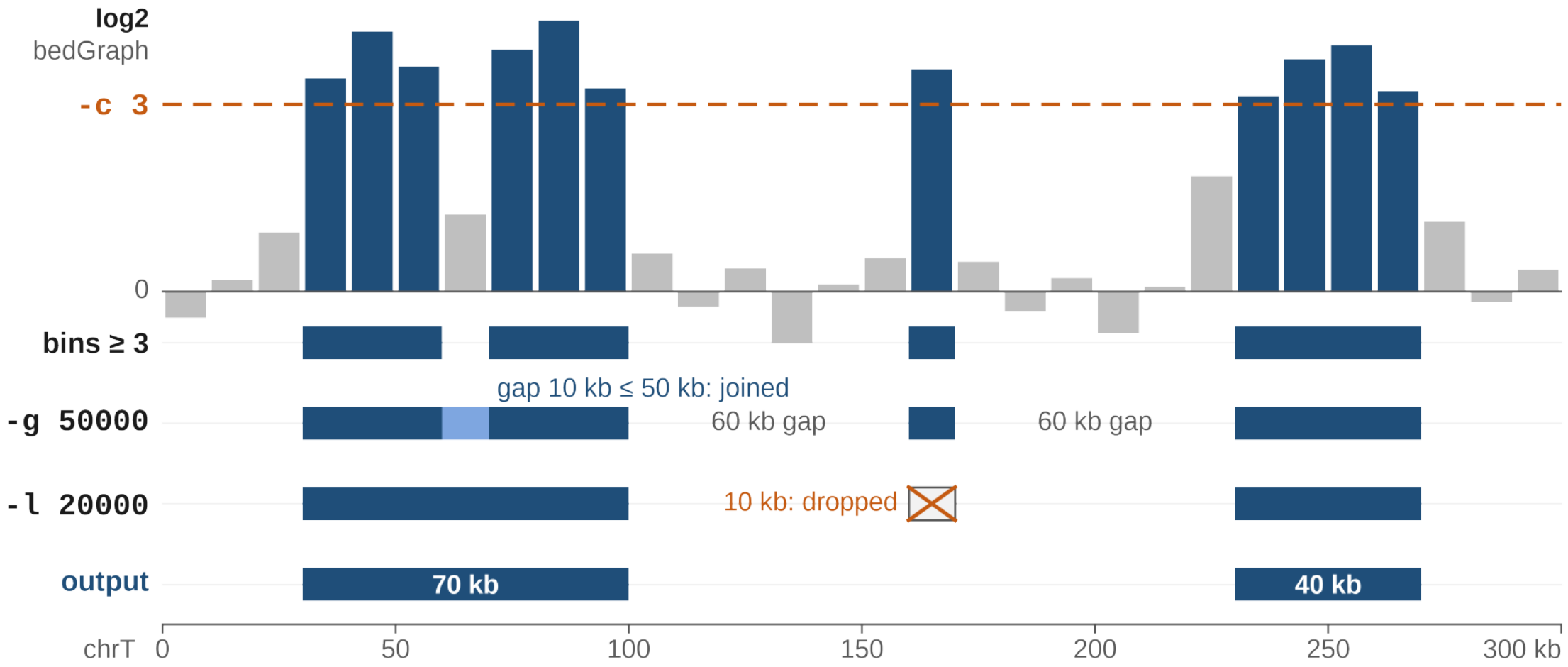


Luger et al. 1997, Nature 389:251 (nucleosome core 147 bp)

Why MACS3 breaks a domain into pieces



bdgpeakcall: cutoff, gap, length



The domain file

```
1     2        3                          4                                5   6  7  8  9     10
track type=narrowPeak name="toy_domains.narrowPeak" description="toy_domains.…
chrT  30000    100000   toy_domains.narrowPeak_narrowPeak1   43   .  0  0  0  55000
chrT  230000   270000   toy_domains.narrowPeak_narrowPeak2   39   .  0  0  0  25000
```

chrom start end name score strand 7-9: always 0 summit

header line
for genome browsers

```
student@vps: ~/chipseq_cenh3

$ tail -n +2 toy_domains.narrowPeak | cut -f 1-3 > toy_domains.bed
$ cat toy_domains.bed
chrT    30000   100000
chrT    230000  270000
$ bedtools getfasta -fi toy.fa -bed toy_domains.bed -fo toy_domains.fa
$ seqkit fx2tab -n -l toy_domains.fa
chrT:30000-100000       70000
chrT:230000-270000      40000
```

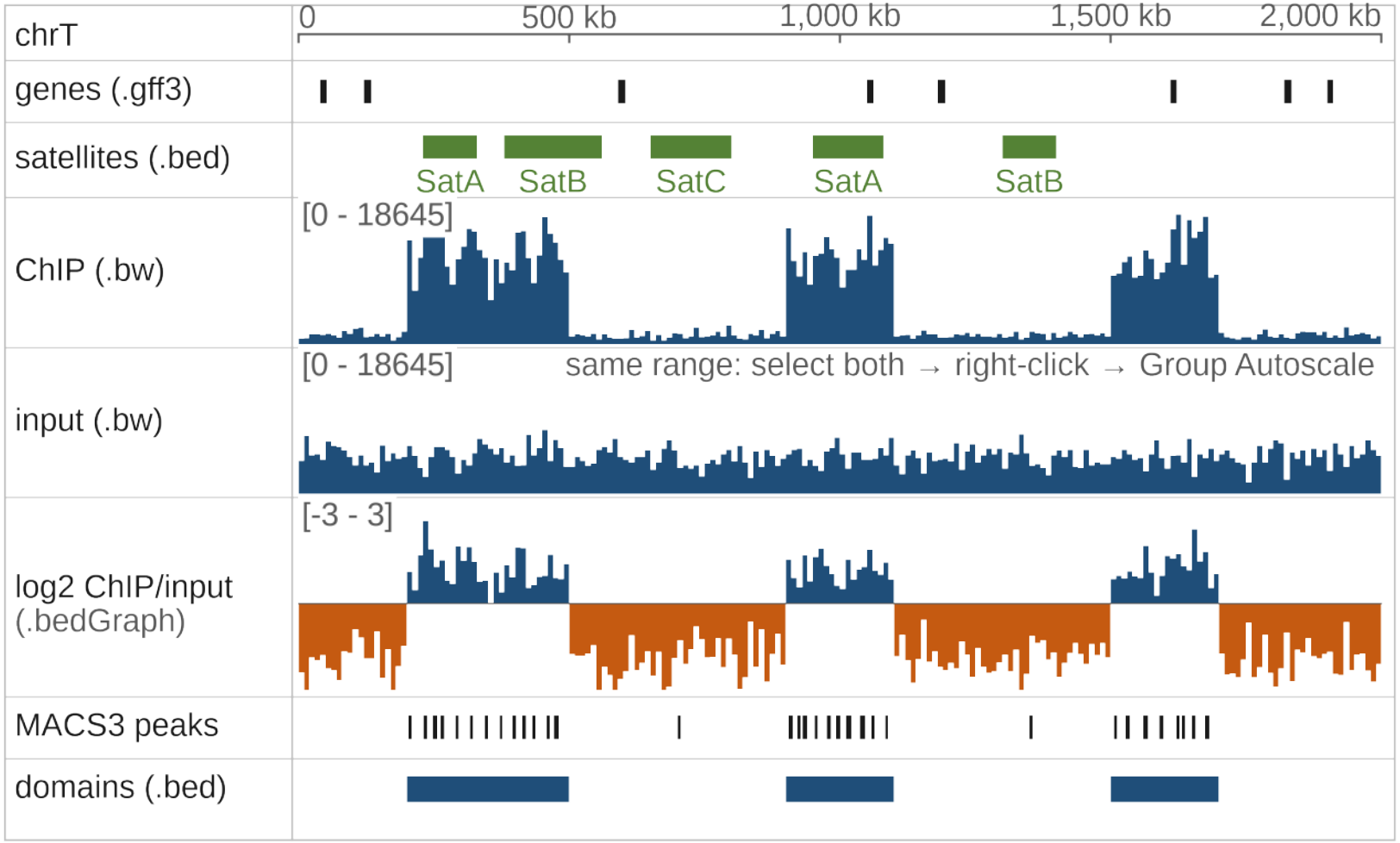
column 5 (score)
= 10 × highest log2 value in the domain
`10 × 4.35 → 43`

column 10 (summit)
middle of the highest bin − start
`85000 − 30000 = 55000`

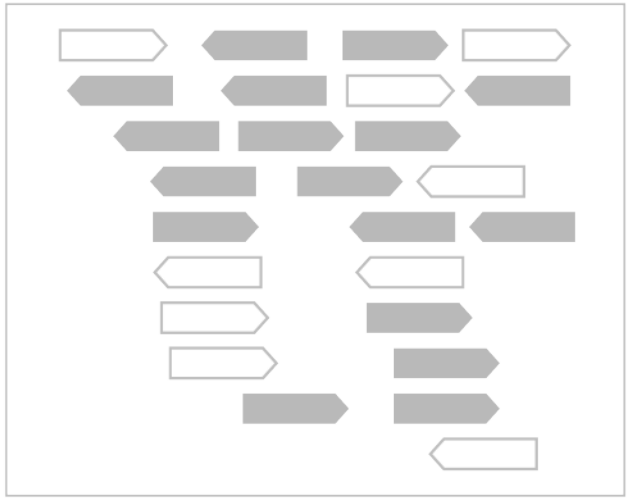
tail -n +2: from line 2
cut -f 1-3: columns 1-3

IGV: one chromosome, many tracks

File → Load from File... (one track per file)



reads (.bam), 600 bp



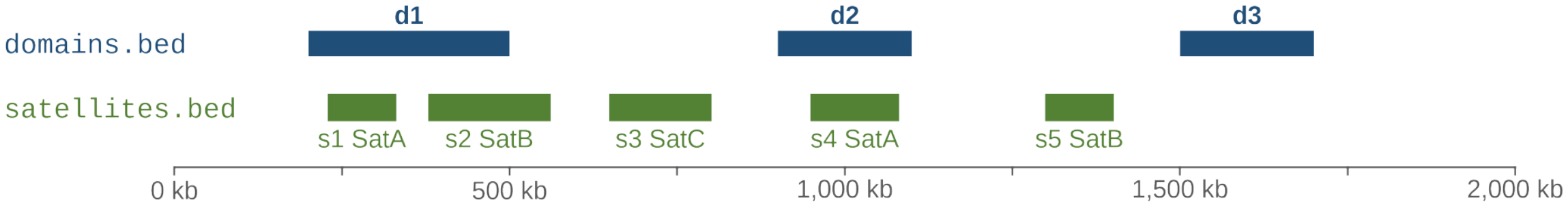
MAPQ > 0

MAPQ 0: hollow

IGV draws MAPQ 0 reads transparent (white fill)

Part 2 – What is inside the CENH3 domains?

bedtools intersect: which regions overlap?




```
bedtools intersect -a domains.bed -b satellites.bed -wa -wb
  chrT  200000  500000   chrT  230000  330000   SatA
  chrT  200000  500000   chrT  380000  560000   SatB
  chrT  900000  1100000  chrT  950000  1080000  SatA
```

one line per overlapping pair: domain + array

d1 × s1
d1 × s2
d2 × s4

d3 overlaps no array → not in the output

```
bedtools intersect -a satellites.bed -b domains.bed -u
  chrT  230000  330000   SatA   s1
  chrT  380000  560000   SatB   s2
  chrT  950000  1080000  SatA   s4
```

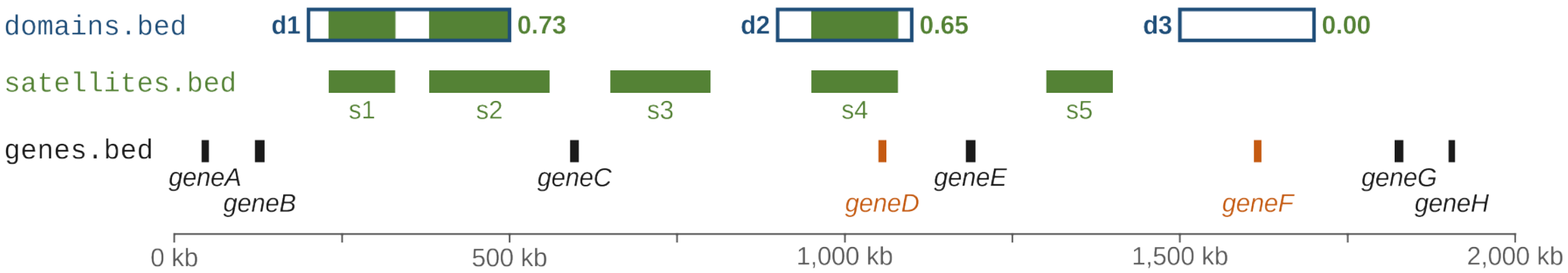
each array touching a domain, once

s2 is only partly inside d1: 1 bp of overlap is enough

```
bedtools intersect -a satellites.bed -b domains.bed -v
  chrT  650000   800000   SatC   s3
  chrT  1300000  1400000  SatB   s5
```

arrays outside all domains

bedtools coverage and genes per Mb




```
bedtools coverage -a domains.bed -b satellites.bed
```

```
chrT   200000   500000   2   220000   300000   0.7333333
chrT   900000   1100000  1   130000   200000   0.6500000
chrT   1500000  1700000  0   0        200000   0.0000000
```

the domain (A) | arrays | bases covered | length of A | fraction

d1 2 arrays cover 220,000 of 300,000 bp
d2 220,000 / 300,000 = 0.73
d3 no array → 0

```
wc -l genes.bed
  8 genes.bed
```

```
bedtools intersect -a genes.bed -b domains.bed -u | wc -l
```

2 genes overlap a domain (geneD, geneF)

genes per Mb

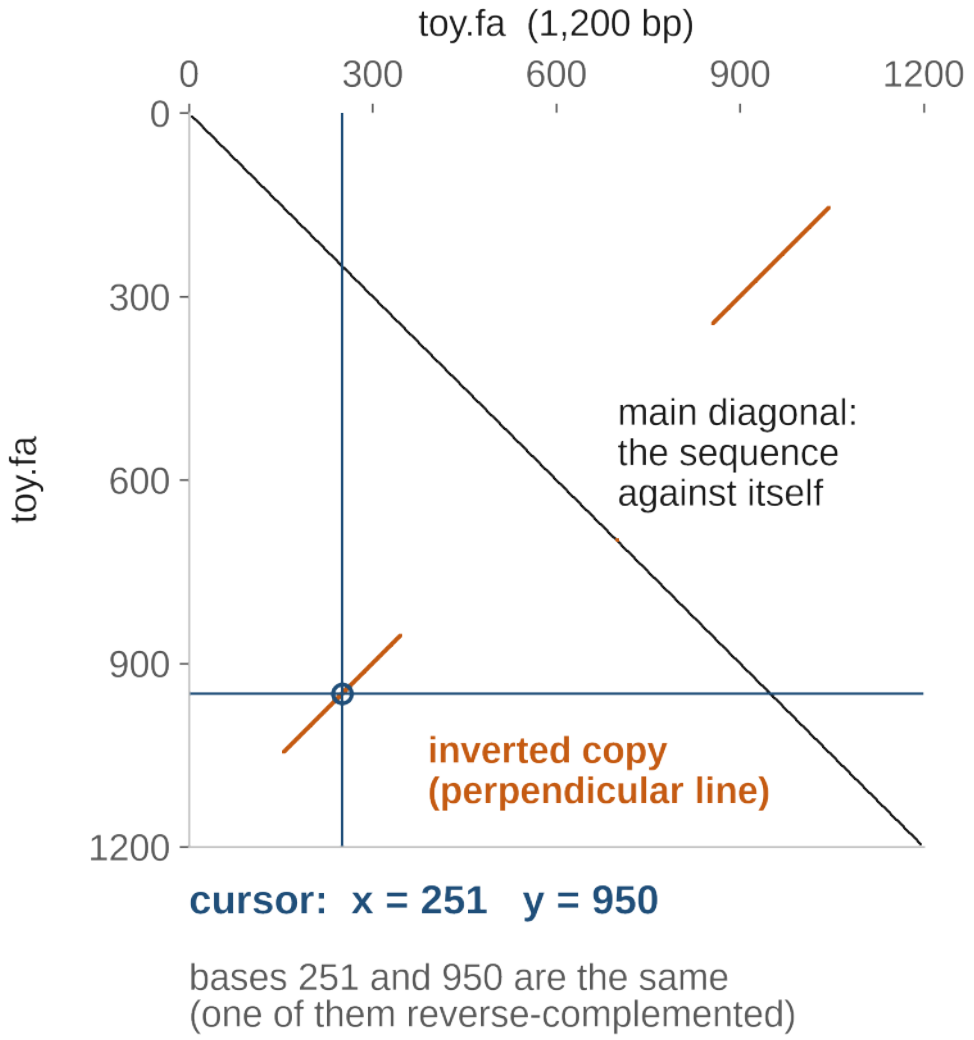
domains: 2 / 0.7 Mb = 2.9

chrT: 8 / 2.0 Mb = 4.0

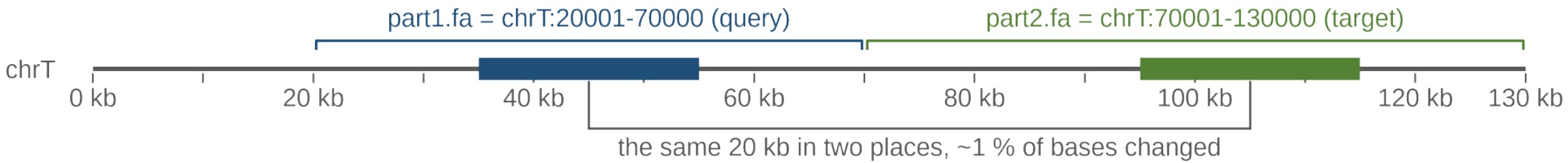
domain length = sum of the 'length of A' column (0.7 Mb)

Dotplots of the arrays: dotter and Gepard

- `samtools faidx`: 20 kb of an array
- `dotter a.fa a.fa &`: against itself
- Move the cursor, read the **positions**
- **Gepard** (Desktop): Mb-long regions; choose sequence 1 and 2



minimap2 for two long sequences: PAF




```
minimap2 -c -x asm10 part2.fa part1.fa | sort -k11,11nr | head -3 | cut -f 1-12
                     target    query
```

longest first,
3 lines, columns 1–12

```
chrT:20001-70000  50000  15000  35004  +  chrT:70001-130000  60000  25000  45001  19780  20015  60
       1            2      3      4     5        6             7      8      9     10     11     12
```

query: name, length, start, end | strand | target: name, length, start, end | matches, aln. length, MAPQ

identity = column 10 / column 11

19780 / 20015 = 0.988

-c = compute the exact alignment
(without it columns 10–11 are only estimates)

positions on chrT

query: 20,000 + 15,000 = 35,000 … 20,000 + 35,004 = 55,004
target: 70,000 + 25,000 = 95,000 … 70,000 + 45,001 = 115,001

PAF counts from 0 inside part1.fa / part2.fa:
add the start of the extracted region − 1

If a step fails

- Read the error; check the file names (`ls`)
- Results of the key steps are in `$D/precomputed`
- Copy them and continue with the next step
- Ask — the same error usually hits several people

1.5 MACS3 failed

```
mkdir -p peaks
cp $D/precomputed/CENH3_macs3_peaks.broadPeak \
  peaks/
```

→ peaks/CENH3_macs3_peaks.broadPeak

↓ 1.6 bdgpeakcall on the bedGraph ↓

1.6 bdgpeakcall failed

```
cp $D/precomputed/CENH3_domains.bed .
cp $D/precomputed/ChIP_vs_input_10kb.bedGraph \
  tracks/
```

→ CENH3_domains.bed
tracks/ChIP_vs_input_10kb.bedGraph

↓ 1.7 IGV, then Part 2 ↓

Part 2 starts from CENH3_domains.bed

check with ls that the files are there